Part A: Object-Oriented Design

MODULE 2: OBJECT-ORIENTED MODELLING



Abu Saleh Musa Miah (PhD)

Presentation Outline

- ✓ **Creating Models in Design**
- ✓ **Major Design Principle**

A model is a simplified representation of a real-world system that shows the important parts of the system and how they relate to each other.

design principlewell-structured, maintainable, reusable, flexible, and understandable

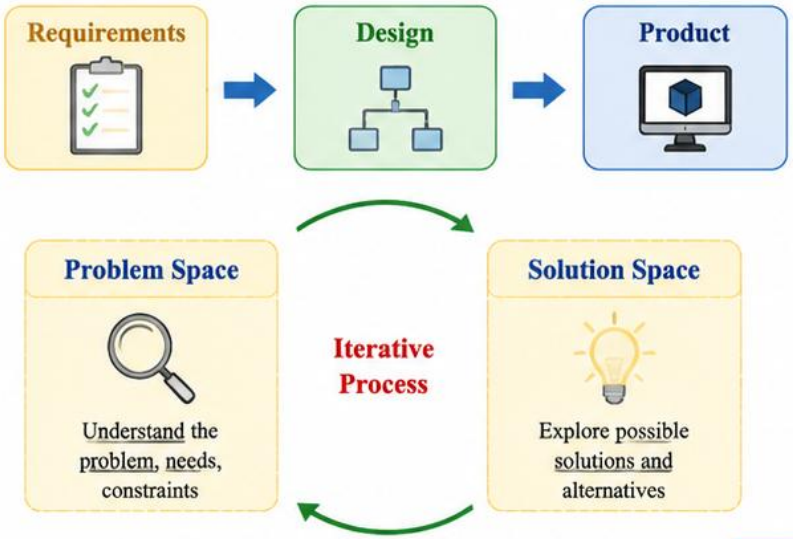
CREATING MODELS IN DESIGN

Design well before you code well

1. DESIGN FIRST, CODE LATER

- Don't jump directly into coding → first understand requirements & design properly
- Design step bridges requirements → product
- Iterative process → deals with problem space & solution space
- Design must be understandable for both users & developers (common terms)
- Object-Oriented Approach helps describe concepts as objects (understood by both users & developers)
- Popular for solving complex problems

Design Process: From Requirements to Product



2. TWO LEVELS OF DESIGN

1 Conceptual Design



- Identify key objects
- Break the problem into manageable parts

2 Technical Design



- Refine object details
- Define attributes (data) and behaviors (operations) for implementation

3. TYPES OF OBJECTS IN DESIGN MODELS

Entity Objects



- Represent things or information in the problem domain
- Focus on the problem space

Control Objects



- Handle events
- Coordinate, sequence and execute actions between other objects
- Control the flow

Boundary Objects



- Interface between the system and external actors or services
- Manage input/output with the outside world

4. ROLE OF MODELS IN SOFTWARE DESIGN



Help organize & understand the design process



Simplify & break down complex problems



Ensure key qualities: reusability, flexibility, maintainability



Serve as design documentation & can map to skeletal code (e.g., Java)

5. UNIFIED MODELLING LANGUAGE (UML)



UML is a visual notation used to create software models.

Different diagrams focus on different issues.

UML Diagram Categories

Structural Models



- Describe the static structure of the system
- Focus on object roles, classes, relationships

Behavioral Models



- Describe how objects interact and behave
- Focus on flow of control, events, use cases

6. ANALOGY IN ARCHITECTURE



Just as architects use scale models to visualize and communicate building designs, **UML models help us visualize, analyse, and communicate software designs.** Models are **abstract** representations that guide effective construction.

Creating Models in Design

- ✓ Don't jump directly into coding → first understand requirements & design properly
- ✓ Design step bridges **requirements → product**
- ✓ Iterative process → deals with **problem space** & **solution space**
- ✓ Design must be understandable for both **users** & **developers** (common terms)
- ✓ **Object-Oriented Approach helps** describe concepts as **objects** (understood by both users & developers)
- ✓ Popular for solving **complex problems**
- ✓ Two levels of design:
  - ✓ **Conceptual Design** → identify key objects, break problem into parts
  - ✓ **Technical Design** → refine object details (attributes & behaviors) for implementation





Creating Models in Design

- ✓ **Types of Objects in Design Models**
  - ➢ **Entity Objects** → focus on problem space
  - ➢ **Control Objects** → handle events & coordinate actions
  - ➢ **Boundary Objects** → connect system to outside services
- ✓ **Role of Models in Software Design**
  - ➢ Help **organize & understand** design process
  - ➢ Simplify & break down complex problems
  - ➢ Ensure qualities: **reusability, flexibility, maintainability**
  - ➢ Serve as **design documentation** & can map to skeletal code (e.g., Java)
- ✓ **Unified Modelling Language (UML)**
  - ➢ Visual notation for software models
  - ➢ Different UML diagrams focus on different issues:
    - ➢ **Structural Models** → describe object roles & relationships
    - ➢ Analogy: like a **scale model in architecture**





Dhaka Urban Management System (DUMS)

Entity Objects, Control Objects & Boundary Objects

1 ENTITY OBJECTS

(Core data objects from the problem space)

Represent real-world things or information that the system manages.

Citizen



- citizenId
- name
- address
- phone
- email

Road



- roadId
- name
- length
- width
- location

Waste Bin



- binid
- location
- capacity
- status
- lastCollectedTime

Building



- buildingId
- type
- address
- floorCount
- ownerId

Traffic Signal



- signalId
- location
- currentState
- cycleTime

2 CONTROL OBJECTS

(Coordinators, logic that controls the flow of events)

Handle requests, make decisions and coordinate between entity objects.

Complaint Controller



- receiveComplaint()
- validateComplaint()
- assignToDepartment()
- updateStatus()

Waste Collection Controller



- scheduleCollection()
- assignWorker()
- updateBinStatus()
- generateReport()

Traffic Management Controller



- monitorTraffic()
- adjustSignalTiming()
- detectCongestion()
- alertAuthority()

Permit Management Controller



- receiveApplication()
- verifyDocuments()
- approveOrReject()
- issuePermit()

3 BOUNDARY OBJECTS

(Interfaces connecting the system with external actors/services)

Manage communication between the system and the outside world.

Mobile App Interface



- sendRequest()
- displayResponse()
- userLogin()

→ Citizens

Web Portal Interface



- submitApplication()
- viewStatus()
- downloadReport()

→ Businesses & Organizations

Sensor / IoT Interface



- receiveSensorData()
- sendCommands()
- deviceStatus()

→ Sensors & IoT Devices

Payment Gateway Interface



- processPayment()
- validateTransaction()
- sendReceipt()

→ Banks / Payment Services

External Agency Interface



- exchangeData()
- receiveUpdates()
- sendReports()

→ Govt. Agencies (Police, DCC, etc.)

How They Work Together

Boundary Objects — Receive requests/data from external actors →

Control Objects — Process requests, make decisions, coordinate →

Entity Objects — Create, update, or retrieve real-world data →

Boundary Objects — Send responses or results back to external actors

Example Scenario

A citizen reports a garbage bin overflow via mobile app (boundary) → Complaint Controller processes it (control) → Waste Bin entity is updated (entity) → Status is sent back to citizen (boundary).

Creating Models in Design: Example

1. Entity Objects (core data objects from the problem space) DUMS

These represent **things in the real world** that your system needs to keep track of.

- ✓ **House Management** → *House, Resident, Water Bill, Electricity Bill, Maintenance Request*
- ✓ **Garbage Management** → *GarbageBin, WasteTruck, CollectionSchedule, DumpingSite*
- ✓ **Traffic Management** → *TrafficLight, Vehicle, RoadSegment, AccidentReport, CongestionLevel*
- ☐ ***Example*: House(id, address, owner, status) or GarbageBin(id, location, capacity, lastCollectedTime)**

2. Control Objects (coordinators, logic that controls the flow of events)

These objects **process requests, make decisions, and coordinate between entity objects**.

- ✓ **House Management** → *BillCalculator, MaintenanceManager,HouseAllocationController*
- ✓ **Garbage Management** → *CollectionScheduler, RouteOptimizer, GarbageMonitoringController*
- ✓ **Traffic Management** → *SignalController, TrafficFlowManager, EmergencyResponseCoordinator*
- ☐ ***Example*:** CollectionScheduler checks all GarbageBin fill levels and sends a task to a WasteTruck

3. Boundary Objects (interfaces connecting the system with external actors/services)

These are **interaction points** with users, devices, or external services.

- ✓ **House Management** → *ResidentAppUI, BillingPortal, SMSNotificationService*
- ✓ **Garbage Management** → *TruckGPSInterface, IoT Sensor in GarbageBin, CitizenComplaintPortal*
- ✓ **Traffic Management** → *DriverMobileApp, CCTV Feed Interface, PoliceDashboard, TrafficL.HI*

💡 *Example*: A *CitizenComplaintPortal* lets residents report uncollected garbage, which is then sent to the CollectionScheduler.

Creating Models in Design: Example

◆ **Iterative & Continuous Design**

- ✓ **First iteration:** define *entity objects* like House, GarbageBin, TrafficLight.
- ✓ **Next iteration:** design *control objects* like BillCalculator, RouteOptimizer, SignalController to manage them.
- ✓ **Then:** refine *boundary objects* like mobile apps, web portals, and IoT sensors that connect the outside world to the system.
- ✓ This cycle continues — as Dhaka grows, new **entities** (e.g., metro stations), new **control logic** (e.g., AI-based traffic prediction), and new **boundary services** (e.g., chatbot for citizens) can be added.

1. Entity Objects (real-world data and resources in the farm) (AAS)

These represent **things being managed** in the farm:

- ✓ **Crop** (type, growth stage, health status)
- ✓ **Soil** (moisture level, pH, nutrient level)
- ✓ **IrrigationPump** (status, capacity)
- ✓ **WeatherData** (temperature, humidity, rainfall)
- ✓ **FertilizerTank** (amount, type)
- ✓ **PestData** (detection, severity)

💡 *Example*: SoilSensor(id, location, moisture, phLevel)

Agricultural Automation System (AAS) – Design Models

Entity Objects, Control Objects & Boundary Objects

1 ENTITY OBJECTS

(Real-world data and resources in the farm)

These represent things being managed in the farm.

Example:
SoilSensor(id, location, moisture, pHLevel)

Crop



- Type
- Growth stage
- Health status

Soil



- Moisture level
- pH level
- Nutrient level

IrrigationPump



- Status
- Capacity
- Location

WeatherData



- Temperature
- Humidity
- Rainfall

FertilizerTank



- Amount
- Type
- Location

PestData



- Detection
- Severity
- Location

2 CONTROL OBJECTS

(Decision-making and process logic)

These coordinate farming tasks based on rules, data, or AI models.

Example:
IrrigationController reads SoilSensor data and activates IrrigationPump if moisture < threshold.

Irrigation Controller



- Decides when & how much to irrigate
- Uses soil moisture, weather & crop data

Fertilizer Scheduler



- Calculatos fertilizer needs
- Based on crop & soil data

Greenhouse Climate Controller



- Controls fan, heater, ventilation
- Maintains temp., humidity

Pest Detection Manager



- Detects pest from sensor/drone data
- Triggers pesticide spraying

Harvest Planner



- Determines best harvest time
- Uses crop maturity & weather forecast

3 BOUNDARY OBJECTS

(Interfaces with users and external services)

These allow interaction between system, farmers, and external tools.

Example:
A FarmerMobileApp shows soil moisture levels and allows manual override of IrrigationController.

Farmer Mobile App



- Monitor crops, soil data, irrigation status
- Manual control & overide

→ Farmers

Weather API Interface



- Get weather forecast
- Temperature, rain, humidity, wind

→ Weather Services

Drone Camera Interface



- Capture crop images
- Detect stress, pests, nutrient deficiency

→ Drones / Image Analysis Service

IoT Sensor Gateway



- Collect data from soil, water, crop sensors
- Send to system

→ Sensors & Field Devices

SMS / Alert Service



- Send alerts about irrigation, pests, low fertilizer, etc.

→ Farmers / Operators

ITERATIVE & CONTINUOUS DESIGN PROCESS

Iteration 1
Model Entity Objects
Identify and model key real-world objects (e.g., Crop, SoilSensor, IrrigationPump)

→

Iteration 2
Design Control Objects
Create logic and rules to manage entities (e.g., IrrigationController, PestDetectionManager)

→

Iteration 3
Design Boundary Objects
Develop interfaces for users and external systems (e.g., FarmerApp, WeatherAPI, DroneInterface)

→

Iteration n
Refine & Optimize
Evaluate, test, and improve models continuously based on feedback and new data

Repeat and Refine for Better Farm Automation

Dhaka Urban Management System (DUMS)

Entity Objects, Control Objects & Boundary Objects

1 ENTITY OBJECTS

(Core data objects from the problem space)

Represent real-world things or information that the system manages.

Citizen
- citizenId
- name
- address
- phone
- email

Road
- roadId
- name
- length
- width
- location

Waste Bin
- binId
- location
- capacity
- status
- lastCollectedTime

Building
- buildingId
- type
- address
- floorCount
- ownerId

Traffic Signal
- signalId
- location
- currentState
- cycleTime

2 CONTROL OBJECTS

(Coordinators, logic that controls the flow of events)

Handle requests, make decisions and coordinate between entity objects.

Complaint Controller
- receiveComplaint()
- validateComplaint()
- assignToDepartment()
- updateStatus()

→ **Waste Collection Controller**
- scheduleCollection()
- assignWorker()
- updateBinStatus()
- generateReport()

→ **Traffic Management Controller**
- monitorTraffic()
- adjustSignalTiming()
- detectCongestion()
- alertAuthority()

→ **Permit Management Controller**
- receiveApplication()
- verifyDocuments()
- approveOrReject()
- issuePermit()

3 BOUNDARY OBJECTS

(Interfaces connecting the system with external actors/services)

Manage communication between the system and the outside world.

Mobile App Interface
- sendRequest()
- displayResponse()
- userLogin()

→ Citizens

Web Portal Interface
- submitApplication()
- viewStatus()
- downloadReport()

→ Businesses & Organizations

Sensor / IoT Interface
- receiveSensorData()
- sendCommands()
- deviceStatus()

→ Sensors & IoT Devices

Payment Gateway Interface
- processPayment()
- validateTransaction()
- sendReceipt()

→ Banks / Payment Services

External Agency Interface
- exchangeData()
- receiveUpdates()
- sendReports()

→ Govt. Agencies (Police, DCC, etc.)

HOW THEY WORK TOGETHER

Boundary Objects — Receive requests/data from external actors →
Control Objects — Process requests, make decisions, coordinate →
Entity Objects — Create, update, or retrieve real-world data →
Boundary Objects — Send responses or results back to external actors

4 ITERATIVE & CONTINUOUS DESIGN

Iteration 1: Define Entity Objects

Identify and define core real-world objects.

House, GarbageBin, TrafficLight, ...

Iteration 2: Design Control Objects

Design the logic and controllers to manage them.

BillCalculator, RouteOptimizer, SignalController, ...

Iteration 3: Refine Boundary Objects

Develop interfaces that connect external users and devices to the system.

Mobile App, Web Portal, IoT Sensor, ...

Continuous Evolution

This cycle continues — as Dhaka grows, new entities (e.g., metro stations), new control logic (e.g., AI-based traffic prediction), and new boundary services (e.g., chatbot for citizens) can be added.

Metro Station, AI Traffic Prediction, Citizen Chatbot, ...

EXAMPLE SCENARIO

A citizen reports a garbage bin overflow via mobile app (boundary) → Complaint Controller processes it (control) → Waste Bin entity is updated (entity) → Status is sent back to citizen (boundary).

Creating Models in Design: Example

2. Control Objects (decision-making and process logic)

These **coordinate farming tasks** based on rules, data, or AI models:

- ✓ **IrrigationController** → decides when and how much to irrigate crops
- ✓ **FertilizerScheduler** → calculates fertilizer requirements based on crop and soil data
- ✓ **GreenhouseClimateController** → controls fan, heater, and humidity
- ✓ **PestDetectionManager** → triggers pesticide spraying if pest threshold is crossed
- ✓ **HarvestPlanner** → determines best harvesting time based on crop maturity & weather forecast

💡 *Example*: IrrigationController reads SoilSensor data and activates IrrigationPump if moisture < threshold.

3. Boundary Objects (interfaces with users and external services)

These allow **interaction between system, farmers, and external tools**:

- ✓ **FarmerMobileApp** → lets farmer monitor crops, soil data, and irrigation status
- ✓ **WeatherAPIInterface** → gets weather forecasts from an external service
- ✓ **DroneCameraInterface** → captures crop health images
- ✓ **IoTSensorGateway** → collects data from soil, water, and crop sensors
- ✓ **SMS/AlertService** → sends alerts about irrigation, pest outbreak, or low fertilizer

💡 *Example*: A *FarmerMobileApp* shows soil moisture levels and allows the farmer to manually override the IrrigationController.

(Iterative & Continuous Design)

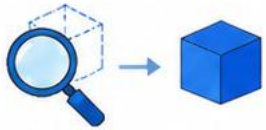
- ✓ **First iteration:** model *entities* (Crop, SoilSensor, IrrigationPump)
- ✓ **Next iteration:** design *control objects* (IrrigationController, PestDetectionManager) to manage entities
- ✓ **Then:** refine *boundary objects* (FarmerApp, DroneInterface, WeatherAPI) for user/system interaction

Object-Oriented Programming: Modeling Objects in a System

Four Key Principles of OOP

1 Abstraction

Show essential features, hide unnecessary details



Focus on what matters, ignore what doesn't.

2 Encapsulation

Bundle data and methods together, hide internal state



Controlled access through methods

3 Decomposition

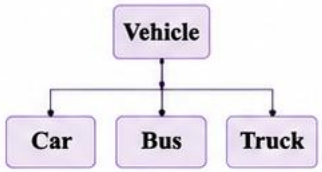
Break a complex system into smaller objects/modules



Easier to design, build and maintain

4 Generalization

Capture common features and create hierarchies



Reuse common structure and behavior

Abstraction: Focus on Essentials, Hide Complexity

What is Abstraction?

- Hides complex implementation details and shows only the essential features of an object.
- Simplifies concepts by focusing on what is important.
- Ignores irrelevant characteristics → follows rule of least astonishment.

Example: Lion



Essential Attributes
- Age
- Species
- Weight

Essential Behaviors
- Hunting
- Eating
- Sleeping

Benefits of Abstraction

- Simplifies class design
- Makes code more focused, understandable & maintainable
- Adaptable when purpose or problems change
- Simplifies complex concepts to essentials within a context

How Abstraction is Implemented

- Implemented via classes; objects are instances of a class.
- Context matters: abstraction depends on software purpose (e.g., gamer vs. athlete).
- Applied at design level using UML class diagrams.
- CRC cards capture components, later refined into classes/functions.

Abstraction Example 1: Dhaka Urban Management System (DUMS)

Step 1: Identify Real-World Concepts

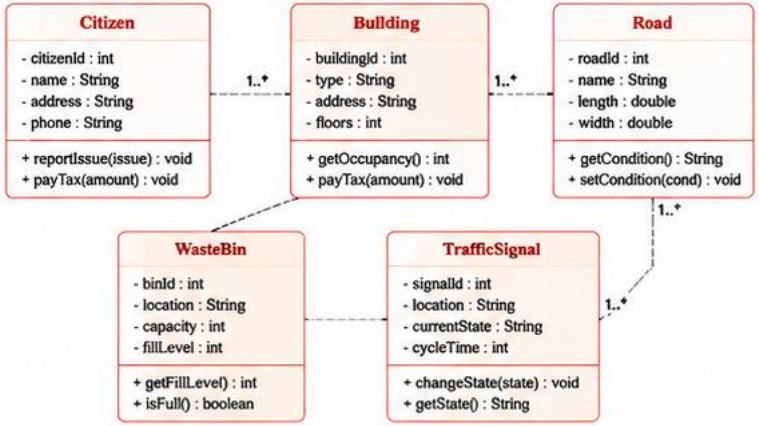
Citizen	People living in Dhaka city
Building	Residential / Commercial building
Road	City roads and streets
Waste Bin	Garbage bins placed around the city
Traffic Signal	Signals controlling traffic flow

Step 2: Capture with CRC Cards

Citizen	Building	Road
Responsibilities Report issue Pay tax Follow rules	**Responsibilities** Provide space Consume utilities Pay tax	**Responsibilities** Enable movement Connect areas Require maintenance
Collaborators Authority, Service	**Collaborators** Citizen, UtilityDept	**Collaborators** TransportDept, Citizen

Waste Bin	Traffic Signal
Responsibilities - Collect waste Indicate fill level	**Responsibilities** - Control traffic Ensure safety
Collaborators WasteDept	**Collaborators** TrafficDept, Road

Step 3: Refine into Classes (UML Class Diagram)



Key Takeaway

Abstraction helps us model real-world complexity by focusing on essential attributes and behaviors of objects.

Identify real-world concepts → Capture using CRC cards → Refine into classes → Relate classes in UML → Implement in code

Evolution of Programming Languages

◆ Language = A system for communicating thoughts and ideas with each other.

✓ System for **communicating ideas** (speaking, writing, gestures, etc.)

✓ Languages must **evolve** to stay alive and useful

✓ Programming languages evolve for:

- ➢ Solving new problems
- ➢ Offering more effective solutions
- ➢ Supporting new data structures

✓ Evolution of ideas → shifts in **programming paradigms**

✓ **Top-Down Programming (Example)**

- ➢ **Design strategy** for solving **data-processing problems**
- ➢ Maps processes → routines (starting from the "top")
- ➢ Expressed as a **tree of routines**
- ➢ Implemented in languages supporting **subroutines**





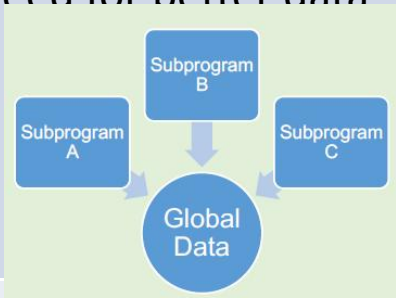
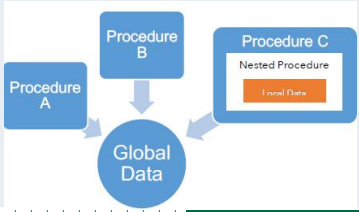
Evolution of Programming Languages

- ✓ **Why Study the History of Programming Paradigms?**
  - Older systems still use **legacy languages & paradigms**
  - Object-oriented programming ≠ best solution for all problems
  - New languages may:
    - Expand on existing structures
    - Modify older paradigms
  - Understanding history helps:
    - Recognize "old in new" paradigms
    - Apply the most **efficient paradigm** for each problem

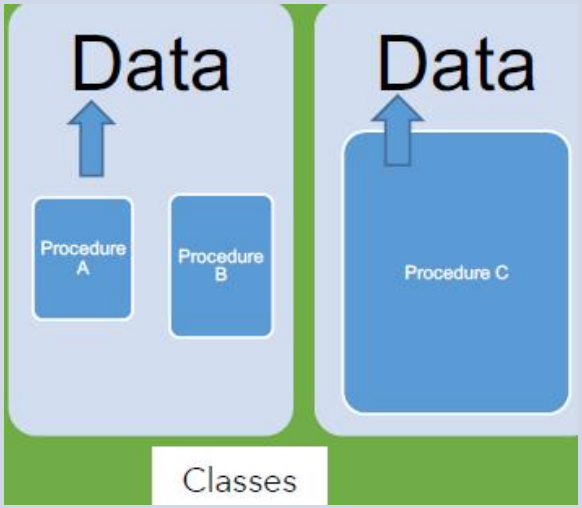




Evolution of Programming Languages

Programming Language and Time period	Solutions afforded by Programming Language	**Uncontrolled global data changes cause unpredictable errors, highlighting the need for better data management.**
COBOL **Fortran** 1960s	COBOL and Fortran employed a centralized global data model that stored information in a single shared memory location, reducing the overhead of passing individual copies through subroutines and enabling faster, more direct access on costly hardware.	Uncontrolled global data changes cause unpredictable errors, highlighting the need for better data management. 
Algol 68 **Pascal** Early 1970s	**Data Management in the 1960s** Global data widely used → changes caused issues for subroutines Solution: **Scopes & local variables** → each subroutine/procedure had its own data. **Abstract Data Types (ADT)** Defined by programmer, not built into language	By the mid-1970s, cheaper computer processing and expensive human labor led to larger, more complex software, making single-file program maintenance difficult. 

Evolution of Programming Languages

Programming Language and Time period	Solutions afforded by Programming Language	**Uncontrolled global data changes cause unpredictable errors, highlighting the need for better data management.**
Object-Oriented Programming (Java, C++, C#, etc.) 1980s to present	**Rise of Object-Oriented Design** Abstract data types improved program management, but **no inheritance existed** Object-oriented design emerged to solve this limitation **Objectives of Object-Oriented Design:** Simplify writing abstract data types Structure systems around **classes** (abstract data types) Enable **inheritance** → one data type can extend another	Object oriented programming is the predominant **programming paradigm** now. 

Evolution of Programming Languages

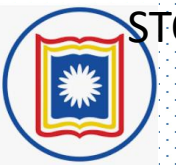
1. COBOL (Business-oriented, verbose English-like, 1960s)

Designed for **business data processing**. DUMS Code looks like English.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GARBAGE-MANAGEMENT.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 BIN1 PIC 9(4) VALUE 120.
01 BIN2 PIC 9(4) VALUE 150.
01 BIN3 PIC 9(4) VALUE 100.
01 TOTAL PIC 9(5).

PROCEDURE DIVISION.
    ADD BIN1 TO TOTAL.
    ADD BIN2 TO TOTAL.
    ADD BIN3 TO TOTAL.
    DISPLAY "TOTAL GARBAGE COLLECTED: " TOTAL.
    STOP RUN.
```

```
01 BIN1 PIC 9(4) VALUE 120.
*> can store numbers like 45, 9999, 0
01 BIN2 PIC 9(2) VALUE 12.
*> can store up to 2 digits (00–99)
01 BIN3 PIC X(10) VALUE "Dhaka". *> X means alphanumeric, up to 10 chars
```





Evolution of Programming Languages

2. ALGOL 68 (Algorithmic, structured programming).

```
begin
int bin1 := 120, bin2 := 150, bin3 := 100;
 int total := bin1 + bin2 + bin3;
print(("TOTAL GARBAGE COLLECTED = ", total))

end
```

3. FORTRAN (Scientific/Engineering, 1950s–1960s)

```
PROGRAM GARBAGE
    INTEGER BIN1, BIN2, BIN3, TOTAL
    BIN1 = 120
    BIN2 = 150
    BIN3 = 100
    TOTAL = BIN1 + BIN2 + BIN3
    PRINT *, 'TOTAL GARBAGE COLLECTED = ', TOTAL
    END
```

4. Pascal (Teaching/Struct Programming, early 1970s)

```
program GarbageManagement;
var
   bin1, bin2, bin3, total: integer;
begin
   bin1 := 120;
   bin2 := 150;
   bin3 := 100;
   total := bin1 + bin2 + bin3;
   writeln('TOTAL GARBAGE COLLECTED = ',
total);
end.
```

Evolution of Programming Languages

3. JAVA

```
class GarbageBin {
    int weight;
    GarbageBin(int w) { weight = w; }
}

public class DhakaUrbanSystem {
    public static void main(String[] args) {
        GarbageBin b1 = new GarbageBin(120);
        GarbageBin b2 = new GarbageBin(150);
        GarbageBin b3 = new GarbageBin(100);

        int total = b1.weight + b2.weight + b3.weight;
        System.out.println("TOTAL GARBAGE COLLECTED = " + total);
    }
}
```





Evolution of Programming Languages

- ✓ **COBOL** → relies heavily on **global data areas** (like *WORKING-STORAGE SECTION*), which makes large projects difficult to manage (hard to avoid conflicts).
- ✓ **ALGOL 68** → introduced **block structure with scopes** (local vs global variables).
- ✓ **Java / OOP** → solves the "global data" problem by **encapsulation**:
  - ➢ Data is stored **inside objects (classes)**
  - ➢ Only accessible through **methods** (controlled access)
  - ➢ Prevents unwanted global data sharing

- ✓ COBOL → everything is global (messy in big systems).
- ✓ ALGOL 68 → introduced **local scopes**.
- ✓ Java/OOP → goes further by **encapsulating data inside classes** and **restricting access** with modifiers





Evolution of Programming Languages

- **Example: Sum of 10 Numbers in Java (OOP Encapsulation)**

```
class NumberSummer{
        private int[] numbers;   // data is private, not global
        private int total;    // Constructor initializes numbers
        public NumberSummer(int[] nums) {
                numbers = nums;
                 total = 0;    }    // Method to calculate sum

 public int calculateSum() {
                for (int n : numbers) {
                total += n;        }
                 return total;    }}

public class Main {
public static void main(String[] args) {
            int[] nums = {1,2,3,4,5,6,7,8,9,10};
 NumberSummer summer = new NumberSummer(nums);
System.out.println("Sum = " + summer.calculateSum());
}}
```

Four Major Design Principles in OOP

- ✓ Object-oriented programming models objects in a system
- ✓ Four key principles: **Abstraction, Encapsulation, Decomposition, Generalization**



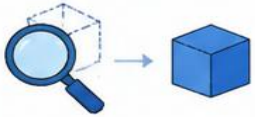


Object-Oriented Programming: Modeling Objects in a System

Four Key Principles of OOP

1 Abstraction

Show essential features, hide unnecessary details



Focus on what matters, ignore what doesn't.

2 Encapsulation

Bundle data and methods together, hide internal state



Controlled access through methods

3 Decomposition

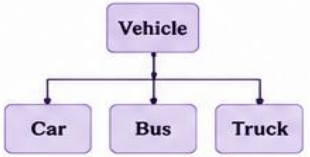
Break a complex system into smaller objects/modules



Easier to design, build and maintain

4 Generalization

Capture common features and create hierarchies



Reuse common structure and behavior

Abstraction: Focus on Essentials, Hide Complexity

What is Abstraction?

- Hides complex implementation details and shows only the essential features of an object.
- Simplifies concepts by focusing on what is important.
- Ignores irrelevant characteristics → follows rule of least astonishment.

Example: Lion



Essential Attributes
- Age
- Species
- Weight

Essential Behaviors
- Hunting
- Eating
- Sleeping

Benefits of Abstraction

- Simplifies class design
- Makes code more focused, understandable & maintainable
- Adaptable when purpose or problems change
- Simplifies complex concepts to essentials within a context

How Abstraction is Implemented

- Implemented via classes; objects are instances of a class.
- Context matters: abstraction depends on software purpose (e.g., gamer vs. athlete).
- Applied at design level using UML class diagrams.
- CRC cards capture components, later refined into classes/functions.

Abstraction Example 1: Dhaka Urban Management System (DUMS)

Step 1: Identify Real-World Concepts

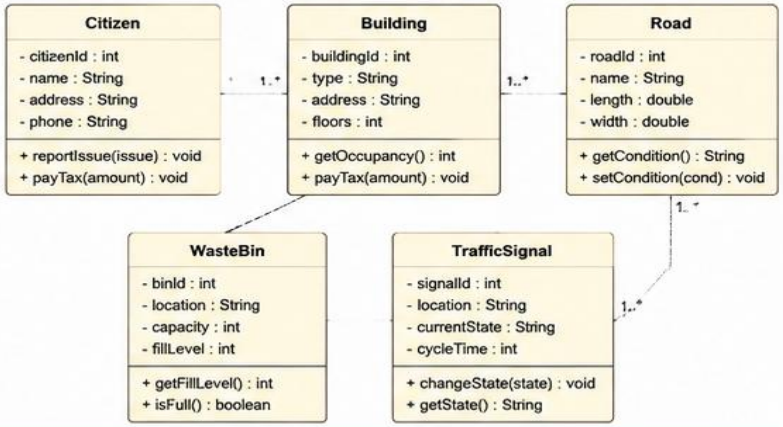
Citizen	People living in Dhaka city
Building	Residential / Commercial building
Road	City roads and streets
Waste Bin	Garbage bins placed around the city
Traffic Signal	Signals controlling traffic flow

Step 2: Capture with CRC Cards

Citizen	Building	Road
Responsbilities Report issue Pay tax Follow rules	Responsbilities Provide space Consume utilities Pay tax	Responsbilities Enable movement Connect areas Require maintenance
Collaborators Authority, Service	Collaborators Citizen, UtilityDept	Collaborators TransportDept, Citizen

Waste Bin	Traffic Signal
Responsbilities Collect waste Indicate fill level	Responsbilities Control traffic Ensure safety
Collaborators WasteDept	Collaborators TrafficDept, Road

Step 3: Refine into Classes (UML Class Diagram)



Key Takeaway

Abstraction helps us model real-world complexity by focusing on essential attributes and behaviors of objects.

Identify real-world concepts → Capture using CRC cards → Refine into classes → Relate classes in UML → Implement in code

Abstraction

- ✓ Abstraction is the process of hiding complex implementation details and showing only the essential features of an object.
- ✓ Simplifies complex concepts by focusing on **essential details**
- ✓ Ignores irrelevant characteristics → follows **rule of least astonishment**
- ✓ Implemented via **classes**; objects are instances of a class
- ✓ **Context matters**: abstraction depends on software purpose (e.g., gamer vs. athlete)
- ✓ Captures two main aspects:
  - ➢ **Basic attributes** (e.g., age of a lion)
  - ➢ **Basic behaviors/responsibilities** (e.g., hunting, eating, sleeping)
- ✓ Benefits:
  - ➢ Simplifies class design
  - ➢ Makes code more **focused, understandable, and maintainable**
  - ➢ Adaptable when system purpose or problem changes





Abstraction

- ✓ Hiding complex reality while exposing only essentials

```
// Abstract class example
abstract class Vehicle { abstract void startEngine(); // Abstract method (no implementation)
 public void honk() { // Concrete method
System.out.println("Beep beep!"); } }

class Car extends Vehicle { @Override void startEngine()
 { System.out.println("Car engine started with key"); } }

 class ElectricCar extends Vehicle { @Override void startEngine()
{ System.out.println("Electric car started with button"); } }
```





Abstraction

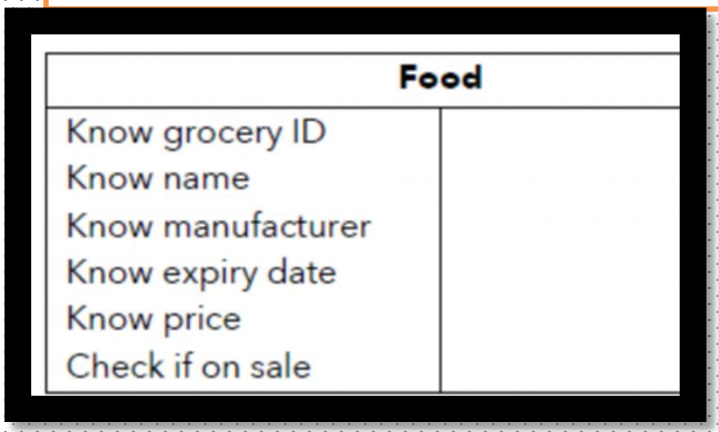
- ✓ Simplifies complex concepts to essentials within a context
- ✓ Applied at the design level using **UML class diagrams**
- ✓ CRC cards capture components, later refined into classes or functions
- ✓ Each concept/class represented as a **box with three sections**:
  - ✓ **Class Name** – same as Java class name
  - ✓ **Properties** – equivalent to Java's member variables
    - ➢ Format: <variable name> : <variable type>
    - ➢ Types can be classes or primitives
  - ✓ **Operations** – equivalent to Java's methods
    - ➢ Format: <method>(<parameter list>) : <return type>

- ✓ **Mapping**:
  - ➢ Class Diagram → Java Class
  - ➢ Properties → Member Variables
  - ➢ Operations → Methods

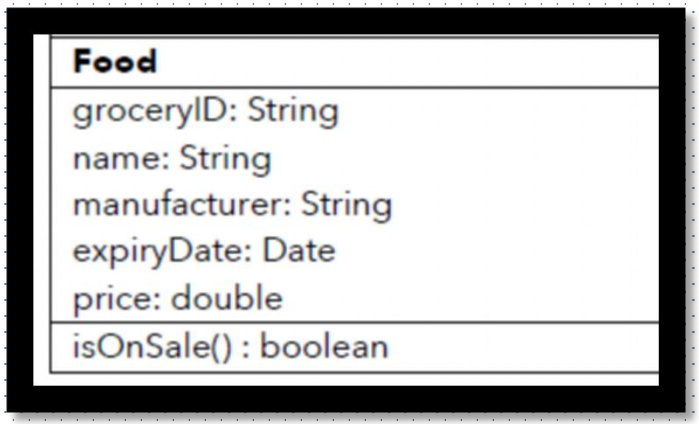




Abstraction



Example: Variables



Example: Class Diagrams

- ✓ **Example: Food Item (Grocery Store Context)**
- ✓ **Class Name**: Food
- ✓ **Properties**: groceryID, name, manufacturer, expiryDate, price
- ✓ **Operation**: isOnSale(date: Date): Boolean
  - ✓ Returns **true/false** depending on sale status

- ✓ **CRC vs. Class Diagram**
  - ➢ CRC: lists responsibilities together
  - ➢ Class Diagram: separates **properties vs. operations**
  - ➢ Easier translation into code for programmers

```
public class Food {
   public String groceryID;
   public String name;
   public String manufacturer;
   public Date expiryDate;
   public double price;
   public boolean isOnSale(Date date) {
      // Implementation
   }}
```

Abstraction

◆ Abstraction Example 1: Dhaka Urban Management System (DUMS)

✓ Suppose we want to model **Garbage Bins** in DUMS: We don't need to care about *sensor chip design* or *truck engine details* → we just need **essential attributes and behaviors**.

```
abstract class GarbageBin {
    // Essential attributes
    protected String location;
    protected int capacity;
    protected int currentLevel;

    // Essential behaviors
    abstract void collectGarbage();
    abstract boolean isFull();
}
```

```
// Concrete class hides details of how collection works
class SmartGarbageBin extends GarbageBin {
    @Override
    void collectGarbage() {
        currentLevel = 0; // reset after collection
        System.out.println("Garbage collected at " +
location);
    }
    @Override
    boolean isFull() {
        return currentLevel >= capacity;
    }}
```

◆ **Abstraction here:**

✓ Shows *location*, *capacity*, and *basic behavior (collectGarbage, isFull)*.

✓ Hides *how sensors detect fullness, how trucks are dispatched, how waste is processed*.

Abstraction

◆ Abstraction Example 2: Agricultural Automation System (AAS)

✓ Suppose we want to model an **Irrigation System**.

```
// Abstract class for irrigation
abstract class IrrigationSystem {
    protected String fieldId;
    protected int waterLevel;

    // Essential behaviors
    abstract void irrigate();
    abstract boolean
needsIrrigation();
}
```

```
// Implementation hides details of pump operation
class SmartIrrigationSystem extends IrrigationSystem
{
    @Override
    void irrigate() {
        waterLevel = 100; // restore soil moisture
        System.out.println("Field " + fieldId + "
irrigated.");
    }
    @Override
    boolean needsIrrigation() {
        return waterLevel < 40; // below threshold
    }
}
```

◆ **Abstraction here:**

✓ Shows fieldId, waterLevel, irrigate, needsIrrigation.Hides how soil sensors measure moisture, how water is pumped, how scheduling is optimized..

Abstraction

- ✓ **Benefits in Both Systems**
  - ➢ **Simplifies design** → Developers think only about key features.
  - ➢ **Maintainability** → If the *garbage collection* logic or *pump mechanism* changes, we don't touch the abstraction.
  - ➢ **Adaptability** → Can extend the system (e.g., add AI-based optimization) without changing high-level abstractions.
- ✓ In **DUMS**, abstraction means thinking of *GarbageBin* with only essential features (location, capacity, collect).
- ✓ In **AAS**, abstraction means thinking of *IrrigationSystem* with only essential features (field, water level, irrigate).

Abstraction Table: DUMS vs AAS

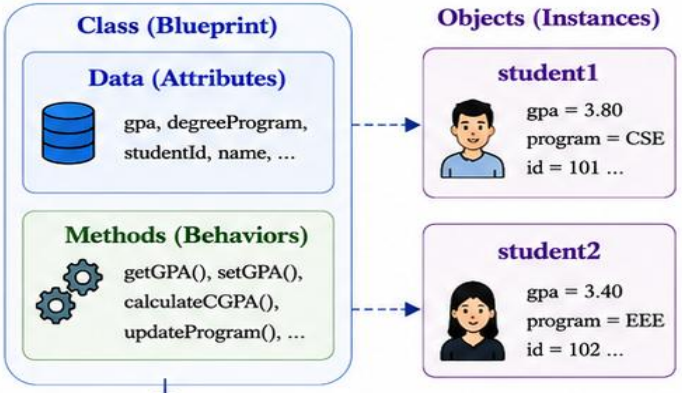
System	Object	Essential Attributes (What it *is*)	Essential Behaviors (What it *does*)
DUMS (Dhaka Urban Management System)	**House**	Address, Owner, Occupants, UtilityStatus	PayBill(), RequestMaintenance(), ReportIssue()
	GarbageBin	Location, Capacity, CurrentLevel	collectGarbage(), isFull(), sendAlert()
	TrafficLight	Location, SignalState (Red/Green/Yellow), Timer	changeSignal(), detectTrafficFlow(), alertEmergency()
AAS (Agricultural Automation System)	**Soil**	Location, MoistureLevel, pH, Nutrients	measureMoisture(), checkFertility()
	Crop	Type, GrowthStage, HealthStatus	grow(), harvest(), detectDisease()
	IrrigationSystem	FieldId, WaterLevel, Status	irrigate(), needsIrrigation(), scheduleWatering()

Encapsulation in Object-Oriented Programming

Encapsulation is the bundling of data (attributes) and the methods (functions) that operate on that data into a single unit (a class).

1. WHAT IS ENCAPSULATION?

- Encapsulation = bundling data (attributes) and methods (behaviors) in a class.
- Controls access to data:
  - Expose what is needed via interfaces
  - Restrict internal data/methods to the object itself
- Ensures each object has its own data values and behaviors.



The class bundles data and methods together.
Each object has its own copy of data.

2. BENEFITS OF ENCAPSULATION

- **Integrity & Security**
  Protects sensitive data and prevents unintended or illegal changes.
- **Changeable Implementation**
  Internal code can change without affecting the interface.
- **Black Box Concept**
  Users interact with the class through methods, without knowing internal details.
- **Modularity & Reusability**
  Simplifies code management and promotes modular design.

Bundles data + functions into a self-contained object, allows controlled access, and restricts direct manipulation of internal data.

3. ENCAPSULATION IN UML (EXAMPLE: STUDENT CLASS)

Student

Attributes (Data)
- gpa : double
- degreeProgram : String
- studentId : int
- name : String

Methods (Operations)
+ getGPA() : double
+ setGPA(gpa : double) : void
+ getProgram() : String
+ setProgram(p : String) : void

Visibility Indicators
+ (public) → accessible from outside the class
- (private) → accessible only within the class

Key Points
- Attributes are private (-) so they cannot be accessed directly from outside.
- Methods are public (+) to provide controlled access to the data.

The private attributes (-) are hidden from outside. Only public methods (+) can access or modify them.

4. GETTER & SETTER METHODS

Getter Methods
(retrieve private data)

Format:
get<AttributeName>()

Example:
getGPA() : double

Setter Methods
(update private data safely)

Format:
set<AttributeName>(value)

Example:
setGPA(double gpa) : void

Getter & Setter methods ensure approved access and modification of data, preserving integrity and security.

SUMMARY: HOW ENCAPSULATION WORKS

Data is hidden (private) inside the class. → Public methods provide controlled access. → Users interact through methods (interface) only. → Data integrity is maintained and system is safe. → Code becomes modular, maintainable and reusable.

Encapsulation in Object-Oriented Programming

- ✓ Encapsulation is the bundling of data (attributes) and the methods (functions) that operate on that data into a single unit (a class).
- ✓ Encapsulation = bundling **data (attributes)** and **methods (behaviors)** in a class
- ✓ Controls access to data:
  - ➢ **Expose** what is needed via interfaces
  - ➢ **Restrict** internal data/methods to the object itself
- ✓ Ensures **each object has its own data values and behaviors**
- ✓ **Benefits of Encapsulation**
  - ➢ **Integrity & Security** protects sensitive data, prevents unintended changes
  - ➢ **Changeable Implementation**: internal code can change without affecting the interface
  - ➢ **Black Box Concept**: users interact with the class through methods, without knowing internal details
  - ➢ **Modularity & Reusability** simplifies code management, promotes modular design





Encapsulation in Object-Oriented Programming UML

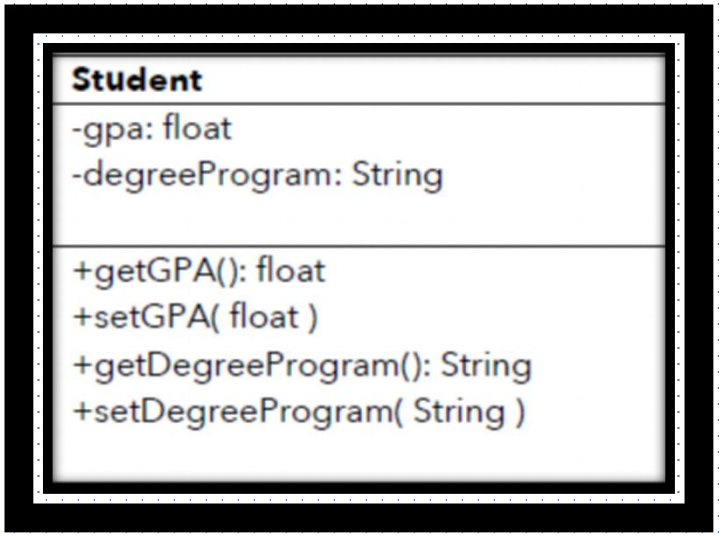
- ✓ Bundles **data + functions** into a self-contained object
- ✓ Allows **controlled access** to data and functions
- ✓ Restricts direct manipulation of internal data
- ✓ Expressed in **UML class diagrams** through attributes + methods
- ✓ **Encapsulation in UML:**
  - ➢ Class diagram **bundles attributes & operations**
  - ➢ Visibility indicators:
    - ✓ **+ (public)** → accessible from outside the class
    - ✓ **- (private)** → accessible only within the class
  - ➢ Example: Student class
    - ✓ Attributes: - gpa, - degreeProgram (private)
    - ✓ Methods: + getGPA(), + setGPA() (public)
- ✓ **Benefits of Encapsulation**
  - ✓ Controls **how data is accessed/modified**
  - ✓ Prevents **unexpected direct changes**
  - ✓ Preserves **data integrity**





Encapsulation in Object-Oriented Programming UML

- **Getter & Setter Methods**
  - ✓ **Getter methods**: retrieve private data
    - ✓ Format: get<AttributeName>()
    - ✓ Example: getGPA()
  - ✓ **Setter methods**: safely update private data
    - ✓ Format: set<AttributeName>(value)
    - ✓ Example: setGPA(double gpa)
  - ✓ Ensure **approved access & modification**



Example: UML Class Diagram for Student

```
public class Student {
    // Private attributes (hidden data)
    private double gpa;
    private String degreeProgram;

    // Public methods (controlled access)
    public double getGPA() {
        return gpa;     }

    public void setGPA(double gpa) {
        if (gpa >= 0 && gpa <= 4.0) {  //
validation
            this.gpa = gpa;
        }
    }
}
```

Encapsulation

UML Visualization:

The full class diagram shows methods and their visibility.

```
----------------------
|     Student      |
----------------------
| - gpa: double      |
| - degreeProgram: String |
----------------------
| + getGPA(): double  |
| + setGPA(gpa): void  |
----------------------
```

```
+-----------------------------+
|           Book              |
+-----------------------------+
| - isbn: String              |
| - title: String             |
| - author: String            |
| - price: Float              |
+-----------------------------+
| + get_isbn(): String        |
| + get_title(): String       |
| + get_author(): String      |
| + get_price(): Float        |
| + set_price(newPrice: Float)|
+-----------------------------+
```

The - denotes private (encapsulated) attributes, and + denotes public methods that form the interface.





Encapsulation: DUMS-Example 1: Garbage Collection

```
public class GarbageBin {
    // Private attributes (hidden data)
    private double fillLevel;    // percentage (0–100)
    private String location;

    // Public methods (controlled access)
    public double getFillLevel() {
        return fillLevel;
    }

    public void updateFillLevel(double level) {
        if (level >= 0 && level <= 100) {
            this.fillLevel = level;
        }
    }

    public String getLocation() {
        return location;
    }
}
```

- ✓ **Encapsulation role:** Prevents outsiders from setting fillLevel to invalid values (e.g., >100%). Only controlled updates are allowed.





Encapsulation: DUMS-Example 2: Traffic Signal Management

```
public class TrafficSignal {
    private String currentColor;   // "Red", "Yellow", "Green"
    private int timer;             // seconds remaining

    public String getCurrentColor() {
        return currentColor;
    }

    public void changeSignal(String color) {
        if (color.equals("Red") || color.equals("Yellow") || color.equals("Green"))
        {
            this.currentColor = color;
        }
    }
}
```

- ✓ **Encapsulation role:** Restricts invalid signals (e.g., "Blue"). Ensures system integrity in city traffic management.





Encapsulation: AAS-Example 1: Soil Sensor

```
public class SoilSensor {
    private double moistureLevel;  // %
    private double pH;             // soil pH
value

    public double getMoistureLevel() {
        return moistureLevel;
    }

    public void setMoistureLevel(double
level) {
        if (level >= 0 && level <= 100) {
            this.moistureLevel = level;
        }
    }

public double getPH() {
    return pH;
}

public void setPH(double pH) {
    if (pH >= 0 && pH <= 14) {
        this.pH = pH;
    }
}
}
```

- ✓ **Encapsulation role:** Protects against invalid sensor values (e.g., negative moisture, impossible pH > 14).

Encapsulation: AAS-Example-2 Irrigation Pump

```
public class IrrigationPump {
    private boolean isOn;
    private int flowRate;   // liters/minute

    public void startPump() {
        isOn = true;
    }

    public void stopPump() {
        isOn = false;
    }

    public void setFlowRate(int rate) {
        if (rate >= 0 && rate <= 100) {
            this.flowRate = rate;
        }
    }

    public boolean getPumpStatus() {
        return isOn;
    }
}
```

- ✓ **Encapsulation role:** Prevents unsafe flow rates and provides safe on/off control.
- ✓ In **DUMS**, encapsulation ensures **public safety & system consistency** (e.g., garbage bins, traffic signals).
- ✓ In **AAS**, encapsulation ensures **data integrity & safe automation** (e.g., soil readings, irrigation controls).

Decomposition in Object-Oriented Design

- ✓ **Definition:** Dividing a whole into smaller parts or combining parts to form a whole
- ✓ **Purpose:** Break problems into manageable pieces, each with a specific responsibility
- ✓ **Relation to Classes:** Each part can be represented by a **class**; the whole object relates to its part classes
- ✓ **Similarity to Abstraction:** Focuses on essential characteristics of each part





Decomposition in Object-Oriented Design

Definition: Dividing a whole into smaller parts or combining parts to form a whole.
Purpose: Break problems into manageable pieces, each with a specific responsibility.

1. NATURE OF PARTS

- **Fixed vs. Dynamic Parts**

Fixed: constant number of parts.
(e.g., oven with 4 burners)

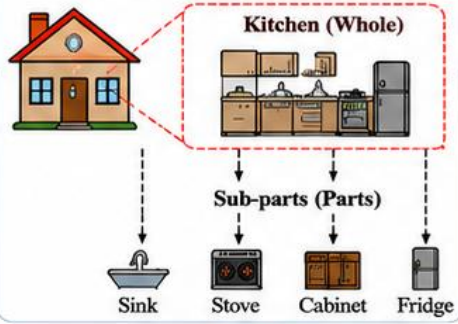


Dynamic: number of parts can change over time.
(e.g., food items in a fridge)



2. PARTS AS WHOLES

A part can itself be a whole with sub-parts.
Example: Kitchen in a House



3. LIFETIMES OF PARTS

Share lifetime with the whole
(e.g., fridge gauge)

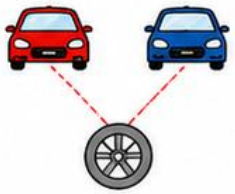


Have independent lifetime
(e.g., food items)



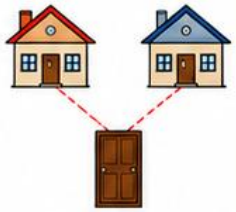
4. SHARED PARTS

Some parts may be shared between wholes.



Wheel (shared)

Some parts cannot be shared.



Door (not shared)

5. RELATION TO CLASSES



Each part can be represented by a class; the whole object relates to its part classes.

5. FOCUS: SIMILARITY TO ABSTRACTION



Focuses on essential characteristics of each part.

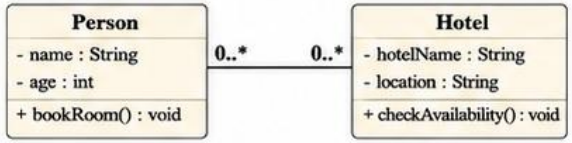
BENEFITS OF DECOMPOSITION

- ✓ Divides a whole into parts (or combines parts into a whole)
- ✓ Helps organize system functionality
- ✓ Improves reusability, maintainability
- ✓ and easier understanding

THREE RELATIONSHIP TYPES IN UML

1. ASSOCIATION (LOOSE RELATIONSHIP)

- Objects interact but are independent.
- If one is destroyed, the other still exists.
- Example: Person ↔ Hotel



A person can interact with many hotels.
A hotel can interact with many people.



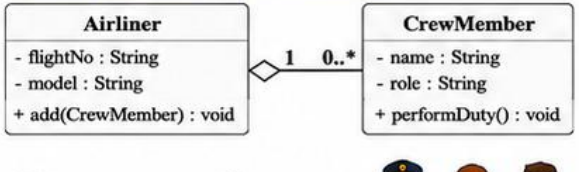
Java Example

```
class Person {
   private String name;
   public void bookRoom(Hotel h) { /* ... */ }
}

class Hotel {
```

2. AGGREGATION (HAS-A RELATIONSHIP, WEAK)

- Whole has parts, but parts can exist independently.
- Example: Airliner ↔ Crew Members





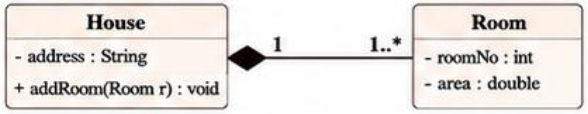
Airliner has a crew.
Crew members exist even if not in airliner.

Java Example

```
public class Airliner {
   private ArrayList<CrewMember> crew = new ArrayList<>();
   public void add(CrewMember crewMember) {
```

3. COMPOSITION (HAS-A RELATIONSHIP, STRONG)

- Whole owns its parts; parts cannot exist without the whole.
- If the whole is destroyed, its parts are also destroyed.
- Example: House ↔ Room





Rooms are part of the house and cannot exist without it.

Java Example

```
public class House {
   private ArrayList<Room> rooms = new ArrayList<>();
```

Decomposition in Object-Oriented Design

- ✓ **Nature of Parts**
- ✓ **Fixed vs. Dynamic Parts:**
  - ➢ Fixed: constant number of parts (e.g., oven with 4 burners)
  - ➢ Dynamic: number of parts can change over time (e.g., food items in a fridge)
- ✓ **Parts as Wholes:** A part can itself be a whole with sub-parts (e.g., kitchen in a house)
- ✓ **Lifetimes:**
  - ➢ Some parts share lifetime with the whole (e.g., fridge gauge)
  - ➢ Some parts have independent lifetimes (e.g., food items)
- ✓ **Shared Parts:** Some parts may be shared between wholes; others cannot





Decomposition in Design

- ✓ Divides a whole into parts (or combines parts into a whole)
- ✓ Helps organize system functionality
- ✓ Three relationship types in UML:
  - ➢ Association
  - ➢ Aggregation
  - ➢ Composition

◆ **Association (Loose Relationship)**

- ✓ Objects interact but are **independent**
- ✓ If one is destroyed, the other still exists
- ✓ Example: **Person ↔ Hotel**
  - ✓ A person can interact with many hotels
  - ✓ A hotel can interact with many people
- ✓ UML: straight line with **0..*** multiplicity
- ✓ Java Example:

```
public class Student {
    public void play(Sport sport) {
        execute.play(sport);
    }
}
```

Example: Student and Sport: separate, but loosely connected



Decomposition in Design

- **Aggregation (Has-a Relationship, Weak)**
  - ✓ Whole has parts, but **parts can exist independently**
  - ✓ Example: **Airliner ↔ Crew Members**
    - ✓ Airliner has a crew
    - ✓ Crew members exist even if not in airliner
  - ✓ UML: empty diamond (◊) at the whole
  - ✓ Java Example:

```
public class Airliner {
   private ArrayList<CrewMember> crew = new
ArrayList<>();
   public void add(CrewMember crewMember) {
      crew.add(crewMember);
   }
}
```

Example: Airliner can have zero or more crew members



Decomposition

- ✓ **UML Visualization (Class Diagram with Relationships):**
  This shows the decomposed classes and their relationships.



*The diagram uses association (has-a), aggregation (<>), and multiplicities (1, *) to show how the decomposed parts relate.*

Composition (Strong Has-a Relationship)

- ✓ **Composition (Strong Has-a Relationship)**
  - ➢ **Exclusive containment** of parts (strongest form of decomposition)
  - ➢ Whole **cannot exist without its parts**
  - ➢ If whole is destroyed → parts are destroyed too
  - ➢ Parts are **tightly bound** and accessed only through the whole
- ✓ **Example: House ↔ Room**
  - ➢ A **House** is made up of **Rooms**
  - ➢ If the House is removed, Rooms no longer exist
  - ➢ UML: **filled diamond (◆)** at the whole (House)
  - ➢ Multiplicity: 1..* (House must have one or more Rooms)

Example:

```
public class House {
   private Room room;

   public House() {
      room = new Room();
   }
}
```

- ◆ **Room object** created when House is created
- ◆ Room cannot exist independently → **tightly coupled**



Example: Composition can be represented with a filled-in diamond using UML class diagrams.

Generalization (Inheritance)

- ✓ **Purpose:** Reduces redundancy by capturing shared characteristics in a **parent class** (superclass)
- ✓ **Mechanism:**
  - ➢ Inheritance allows **child classes** (subclasses) to reuse attributes and behaviors from parent classes
  - ➢ Child classes can add specialized attributes or behaviors
- ✓ **Benefits:**
  - ➢ Saves time and prevents errors
  - ➢ Makes software more **flexible, maintainable, and reusable**
  - ➢ Changes in common code only need to be made in the superclass
  - ➢ Supports easy expansion of software with new subclasses

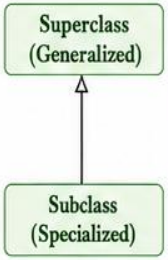




Purpose: Reduces redundancy by capturing shared characteristics in a parent class (superclass).

1. WHAT IS GENERALIZATION?

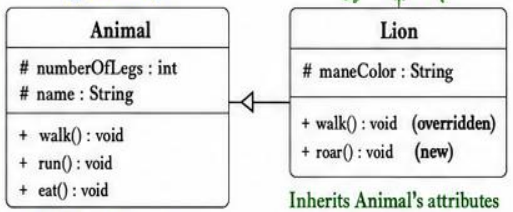
- Factors out shared characteristics into superclass.
- Supports code reuse and avoids redundancy.
- Subclasses inherit superclass' attributes & methods.
- Represented in UML with solid line + arrow (↑).
- Superclass at arrowhead (top), subclass at tail (bottom).



2. EXAMPLE: ANIMAL ↔ LION

Superclass: Animal (parent) (generalized)

Subclass: Lion (specialized)



Declared abstract → cannot be instantiated.

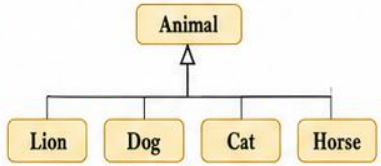
Inherits Animal's attributes & behaviours and adds specialized behaviour.

3. BENEFITS OF GENERALIZATION

- Saves time and prevents errors through code reuse.
- Makes software more flexible, maintainable, and reusable.
- Changes in common code only need to be made in the superclass.
- Supports easy expansion of software with new subclasses.

4. UML VISUALIZATION (INHERITANCE HIERARCHY)

UML uses an arrow with a solid line to represent inheritance (generalization).



Animal is the superclass.
Lion, Dog, Cat, Horse are subclasses.

5. CONSTRUCTORS IN INHERITANCE

Implicit Constructor (default)

Auto-assigned default values (to 0, false, null, etc.)

```
class Animal {
    int numberOfLegs;   // 0
    String name;        // null
}
```

Explicit Constructor (initialization)

Initialize attributes during object creation.

```
class Animal {
    int numberOfLegs;
    String name;

    Animal(int legs, String n) {
        numberOfLegs = legs;
        name = n;
    } }
```

Subclass must call superclass constructor using super().

```
class Lion extends Animal {
    String maneColor;

    Lion(int legs, String n,
         String color) {
        super(legs, n);   // call
        maneColor = color;
    }
}
```

6. METHOD OVERRIDING

Subclass can redefine (superclass) methods.

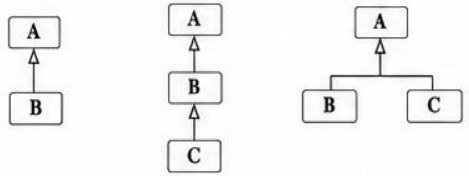
```
class Animal {
    void walk() {
        System.out.println
          ("I walk");
    }
}
```

```
class Lion extends Animal {
    @Override
    void walk() {
        System.out.println
          ("I'd rather nap");
        roar(legs, n);   // call
    }
```

Allows specialized behaviour while keeping inheritance structure.

7. TYPES OF INHERITANCE IN JAVA

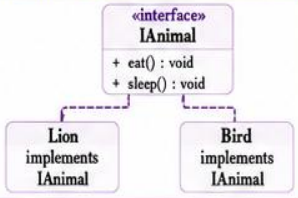
- Java supports single implementation inheritance (using extends).



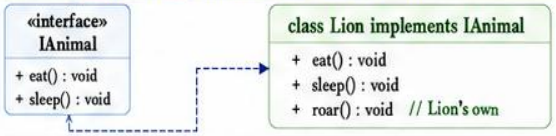
- A superclass can have multiple subclasses.
- Subclasses can also be superclasses (multi-level inheritance).

8. INTERFACE INHERITANCE

- Java alternative to multiple inheritance.
- Declares method signatures only (no attributes or method bodies).
- Implemented using implements keyword.
- UML: interface shown with «interface» and dotted line + hollow arrow (↑).



9. EXAMPLE: LION IMPLEMENTS IANIMAL

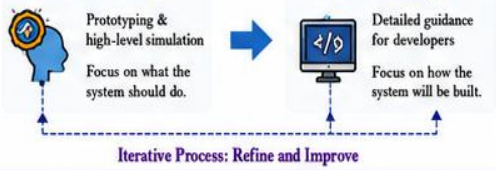


- ✓ Lion behaves like a Lion and like an IAnimal.
- ✓ Multiple interfaces can be implemented → no ambiguity.

10. ADVANTAGES OF INTERFACES

- Enable polymorphism (same behaviour, different implementation).
- Allow multiple inheritance of type without ambiguity.
- Promote flexibility & reuse in system design.

11. DESIGN PROCESS IN OOP



12. DESIGN TOOLS

Conceptual Tool: CRC Cards

- Easy communication with clients.
- Focus on design without writing code.

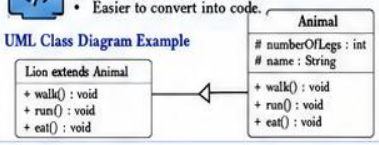
CRC Card Example

Class Name	Responsibilities	Collaborators
Lion	Roar, Eat, Sleep	Animal, Zookeeper

Technical Tool: UML Class Diagrams

- More detailed than CRC cards.
- Easier to convert into code.

UML Class Diagram Example



13. INTEGRATION WITH OOP PRINCIPLES

Abstraction
Focus on essential features of objects.

Encapsulation
Bundle data & methods, control access.



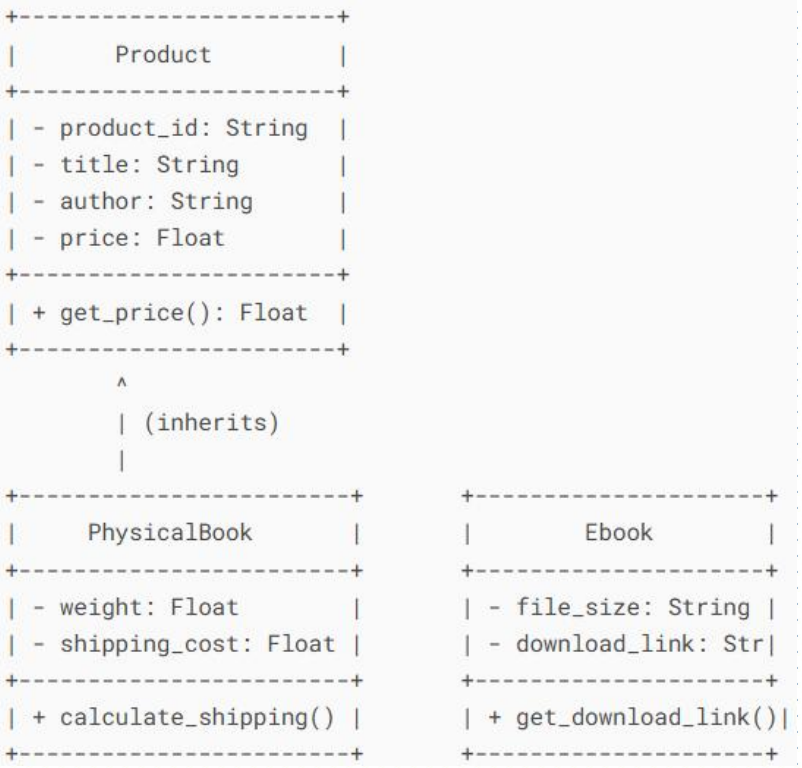
Decomposition
Break system into manageable parts.

Generalization (Shtyping)
Reuse & extend existing classes.

Key Takeaway: Generalization (Inheritance) promotes code reuse and extensibility by modeling "is-a" relationships between classes. It works together with Abstraction, Encapsulation, and Decomposition to build robust, maintainable, and scalable systems.

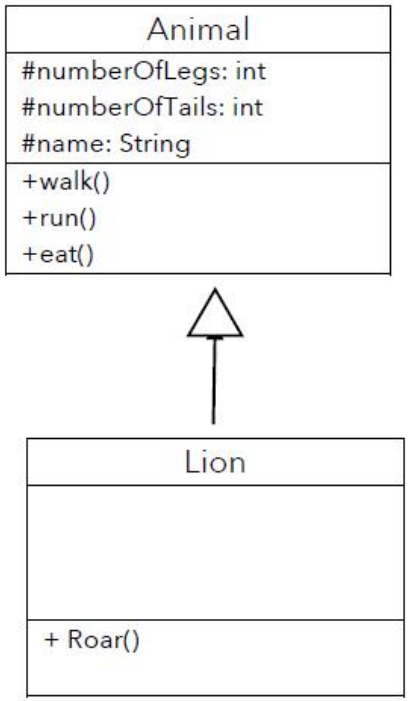
Generalization (Inheritance)

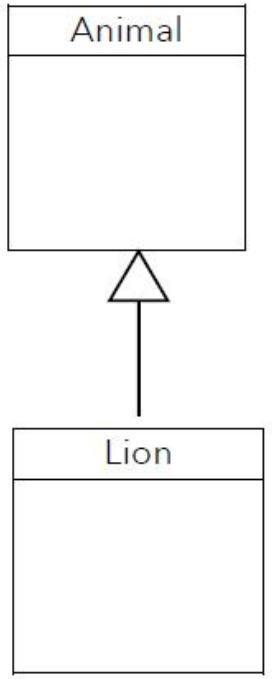
- ✓ **UML Visualization (Inheritance Hierarchy):**
  UML uses an arrow with a solid line to represent inheritance (generalization).



Generalization (Inheritance)

- ✓ Factors out **shared characteristics** into a superclass
- ✓ Supports **code reuse** and avoids redundancy
- ✓ Subclasses inherit superclass' attributes & methods
- ✓ Represented in UML with **solid line + arrow (↑)**
  - ➢ **Superclass** at arrowhead (top)
  - ➢ **Subclass** at tail (bottom)
- ✓ **Example: Animal ↔ Lion**
- ✓ **Superclass**: Animal (generalized)
  - ➢ Attributes: #numberOfLegs, #name, etc.
  - ➢ Methods: walk(), run(), eat()
  - ➢ Declared **abstract** → cannot be instantiated
- ✓ **Subclass**: Lion (specialized)
  - ➢ Inherits Animal's attributes & behaviours
  - ➢ Adds specialized behaviour → roar()





Generalization (Inheritance)

- ✓ **Constructors in Inheritance**
  - ➢ **Implicit constructor**: auto-assigned defaults (0, null)
  - ➢ **Explicit constructor**: initialize attributes during object creation
  - ➢ Subclass must call superclass constructor using **super()**
- ✓ **Method Overriding**
  - ➢ Subclass can **redefine superclass methods**

  Example:
  public void walk() {
  System.out.println("I'd rather nap"); } Allows specialized behaviour while keeping inheritance structure

- ✓ **Types of Inheritance in Java**
  - ➢ **Single implementation inheritance** only (using extends)
  - ➢ A superclass can have multiple subclasses
  - ➢ Subclasses can also be superclasses (multi-level inheritance)





Generalization (Inheritance)

- ✓ **Interface Inheritance**
  - ➢ Java alternative to multiple inheritance
  - ➢ Declares **method signatures only** (no attributes or bodies)
  - ➢ Implemented using **implements keyword**
  - ➢ UML: interface shown with «interface» + **dotted arrow (↑)**

- ✓ Lion behaves like a Lion **and** like an IAnimal
- ✓ Multiple interfaces can be implemented → no ambiguity
- ✓ **Advantages of Interfaces**
  - ➢ Enable **polymorphism** (same behaviour, different implementation)
  - ➢ Allow **multiple inheritance of type** without ambiguity
  - ➢ Promote **flexibility & reuse** in system design

```
public interface IAnimal {
    void move();
    void speak();
    void eat();
}
public class Lion implements IAnimal {
    public void move() { … }
    public void speak()
{ System.out.println("Roar!"); }
    public void eat() { … }
}
```

Example: Animal Inheritance

Design Structure in Java and UML Class Diagrams

- ✓ **Design Process:**
  - ➢ Conceptual design → prototyping & high-level simulation
  - ➢ Technical design → detailed guidance for developers
- ✓ **Conceptual Tools:**
  - ➢ CRC cards → easy communication with clients, focus on design without code
- ✓ **Technical Tools:**
  - ➢ UML Class Diagrams → more detailed than CRC cards, easier conversion to code
- ✓ **Integration with OOP Principles:**
  - ➢ Abstraction, Encapsulation, Decomposition, Generalization applied in Java and UML





Example

```
// Demonstrate method overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for one integer parameter.
  void test(int a) {
    System.out.println("a: " + a);
  }

  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }

  // overload test for a double parameter
  double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
  }
}
```

```
class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    double result;

    // call all versions of test()
    ob.test();
    ob.test(10);
    ob.test(10, 20);
    result = ob.test(123.25);
    System.out.println("Result of ob.test(123.25): " + result);
  }
}
```

Access Control

- Java's access specifiers are **public**, **private**, and **protected**
- Java

Access Levels

Modifier	Class	Package	Subclass	World
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
no modifier	Y	Y	N	N
`private`	Y	N	N	N

Thank You Q/A